

CM52054: Foundational Machine Learning

Parametric & Non-parametric Approximations

Wenbin Li



Recall - linear regression

Given a set of data points (pairs of input and output)

$$\mathcal{D} = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N)\} \subset \mathcal{X} \times \mathcal{Y} \subset \mathbb{R}^M \times \mathbb{R},$$

The goal is to find the best-fitting line that minimises the sum of squared errors (SSE):

$$SSE = \sum_i (\text{Prediction} - \text{Output})^2.$$

The linear function is defined as:

$$f(\mathbf{x}) = w_0x_0 + w_1x_1 + \dots + w_Mx_M = \mathbf{w}^\top \mathbf{x}$$

The optimum solution can be written as:

$$\mathbf{w}_* = \arg \min_{\mathbf{w} \in \mathbb{R}^n} \sum_{i=1}^N (\mathbf{w}^\top \mathbf{x}_i - y_i)^2.$$

Recall - Bayesian linear regression

We define a model with zero mean noise:

$$y_i = \mathbf{w}^\top \mathbf{x}_i + \epsilon_i,$$
$$\epsilon_i \sim \mathcal{N}(0, \sigma^2)$$

It can be rewritten as:

$$y_i - \mathbf{w}^\top \mathbf{x}_i \sim \mathcal{N}(0, \sigma^2),$$

which gives the **likelihood** expression:

$$p(y_i | \mathbf{x}_i, \mathbf{w}) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - \mathbf{w}^\top \mathbf{x}_i)^2}{2\sigma^2}\right)$$

Parametric modelling

What is parametric modelling?

- Assume some functional form of the models given dataset $\mathcal{D}$
- Functional form: Linear, Gaussian, Binomial, Logistic etc.
- Estimate unknown parameters such as w , σ , etc.

Parametric modelling

- Pros:
 - Often faster
 - Size of training data is smaller
 - May work well even if the training data is not fitting perfectly

Parametric modelling

- Pros:
 - Often faster
 - Size of training data is smaller
 - May work well even if the training data is not fitting perfectly
- Cons:
 - Highly constrained to the specific chosen functional form
 - Need to make strong distributional assumptions, often not realistic
 - Difficult to model highly complex problems

Parametric modelling algorithms

Algorithms belongs to parametric type:

- Logistic Regression
- (Naive) Bayesian Regression (refer to Bayesian ML unit)
- Perceptron & MLP (refer to NN lecture)
- Most Monte Carlo sampling methods, e.g., Markov Chain Monte Carlo (MCMC) (refer to Bayesian ML unit)
- Laplace approximation

Non-parametric modelling

Then what is **non**-parametric modelling?

- Typically **don't** make any distributional assumptions
- More data points enable the model to learn more accurate/complex scenarios
- Number of parameters scale with number of training samples

Parametric vs non-parametric: an example



Non-parametric modelling

- **Pros:**
 - **Flexibility:** Capable of fitting a large number of functional forms
 - **Power:** **No assumptions** (or weak assumptions) about the underlying function.
 - **Performance:** Can result in higher performance models for prediction.

Non-parametric modelling

- **Pros:**

- **Flexibility:** Capable of fitting a large number of functional forms
- **Power:** **No assumptions** (or weak assumptions) about the underlying function.
- **Performance:** Can result in higher performance models for prediction.

- **Cons:**

- **More data:** Require a lot more training data to estimate the mapping function
- **Slower:** A lot slower to train as they often have far more parameters to train
- **Overfitting:** More of a risk to overfit the training data and it is harder to explain why specific predictions are made

Non-parametric modelling

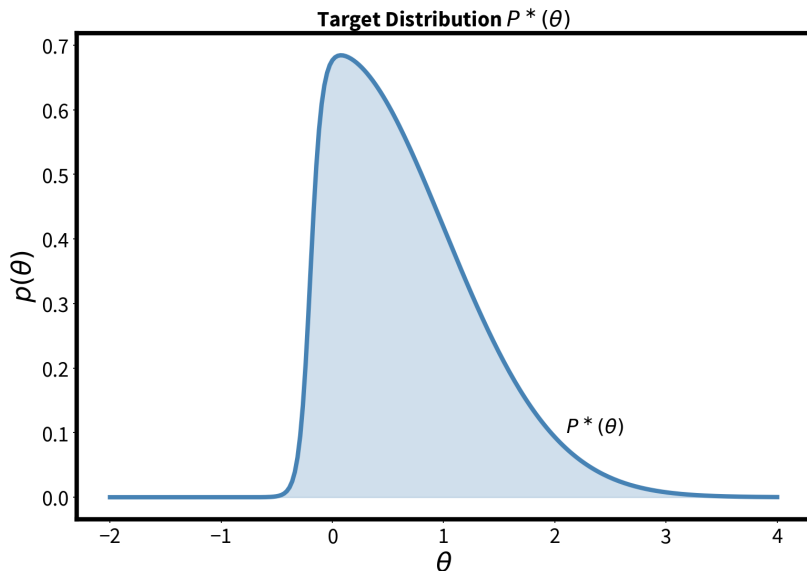
Algorithms belongs to non-parametric type:

- Kernel Density Estimation (KDE)
- K-Nearest Neighbors (kNN)
- Support Vector Machines (SVM)
- Gaussian Processes (GP)

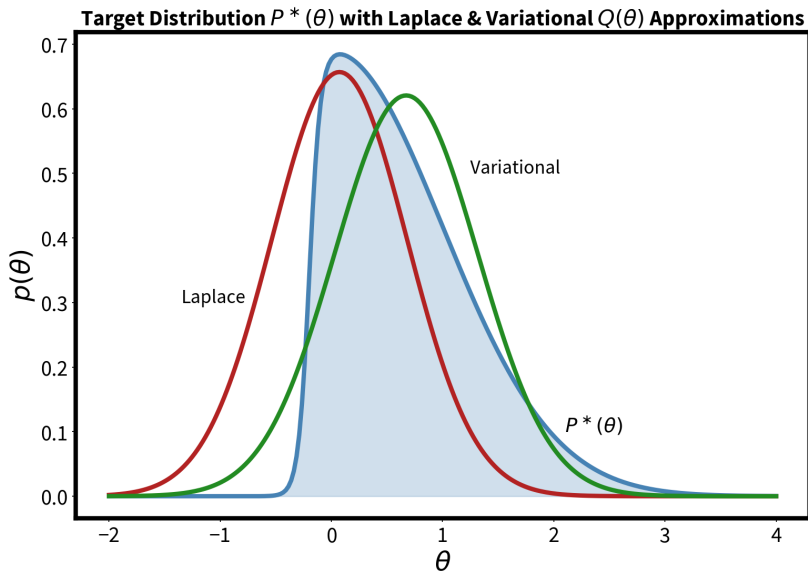
Parametric example: Laplace approximation

- Laplace approximation can be treated as a simplified variational inference (VI) method
- VI aims to **design an analytically known distribution** $Q^*(\theta)$ to **approximate** the target distribution $P^*(\theta)$

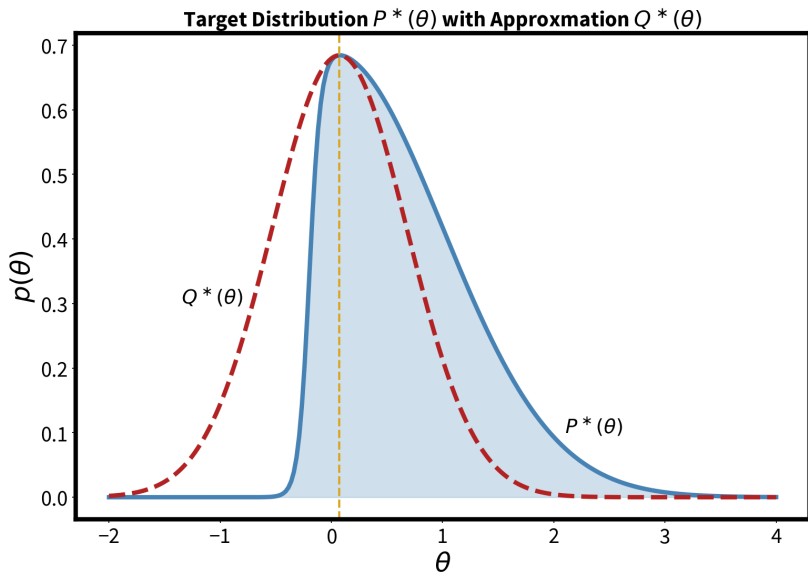
Parametric example: Laplace approximation



Parametric example: Laplace approximation

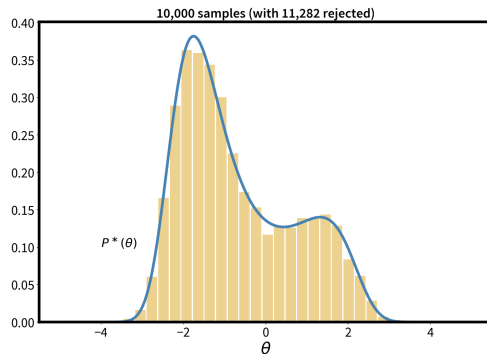


Parametric example: Laplace approximation



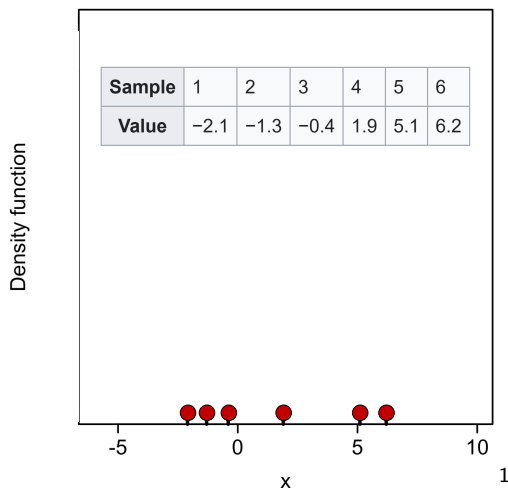
Non-parametric example: Kernel Density Estimation

Kernel Density Estimation is a method to estimate the distribution (probability density) of input data points by binning.



Non-parametric example: Kernel Density Estimation

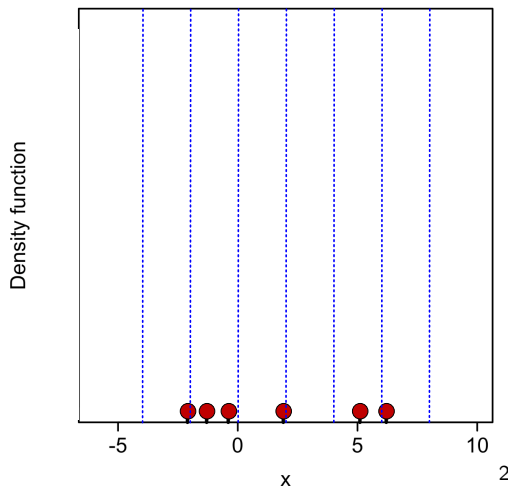
There are 6 samples that follow an unknown distribution.



¹Figure from Wiki.

Non-parametric example: Kernel Density Estimation

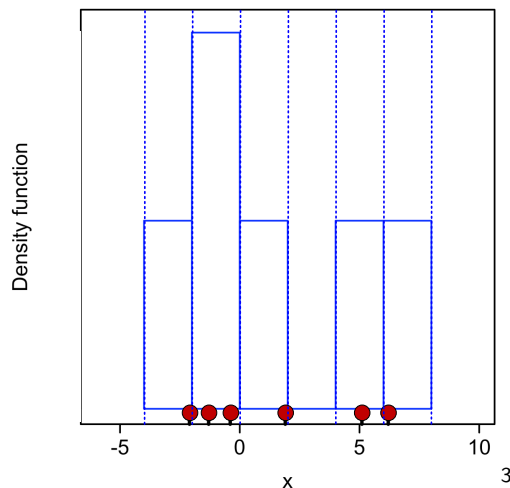
The width of the bin is set to the size of 2 units.



²Figure from Wiki.

Non-parametric example: Kernel Density Estimation

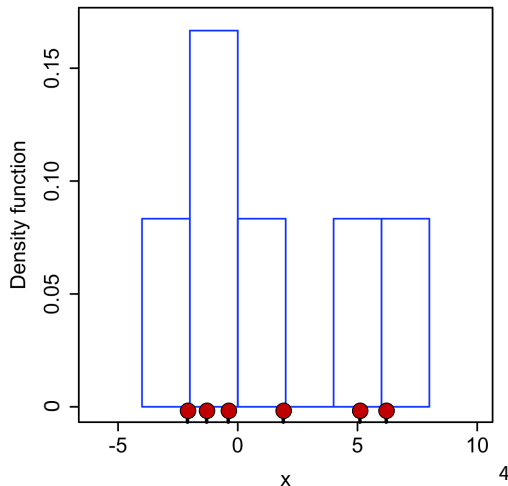
The histogram of samples can be obtained for each bin.



³Figure from Wiki.

Non-parametric example: Kernel Density Estimation

The density value can be obtained by normalisation.



⁴Figure from Wiki.

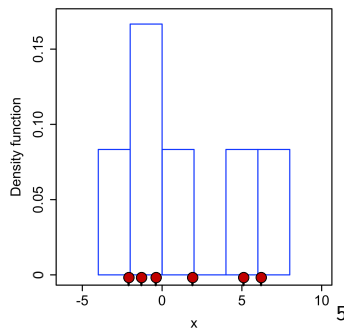
Non-parametric example: Kernel Density Estimation

How do we estimate density in mathematics?

Cumulative Distribution Function (CDF):

$$f(x) = \lim_{h \rightarrow 0} \frac{F(x+h) - F(x-h)}{2h}$$

where h is the width of the bin



Non-parametric example: Kernel Density Estimation

For N data points, the density can be computed by:

$$\tilde{f}_h(x) = \frac{1}{2h} \frac{\#\{x_i \in [x-h, x+h]\}}{N}$$

This could be further summarised as:

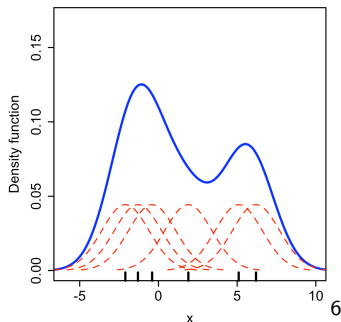
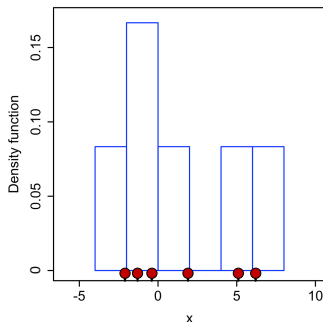
$$\tilde{f}_h(x) = \frac{1}{Nh} \sum_{i=1}^N G\left(\frac{x - x_i}{h}\right)$$

where $G(\cdot)$ is the 'kernel' function in KDE.

Non-parametric example: Kernel Density Estimation

$$\tilde{f}_h(x) = \frac{1}{Nh} \sum_{i=1}^N G\left(\frac{x - x_i}{h}\right)$$

where $G(\cdot)$ is the 'kernel' function in KDE.



⁶Figure from Wiki.

Optional: Gaussian Processes

A **Gaussian process** (GP) defines a **distribution over functions**, $p(f)$ that can be used for Bayesian regression:

$$p(f|D) = \frac{p(f)p(D|f)}{p(D)},$$

where f is a function mapping from some input space to some feature space.

We already have **input data points** $\{\mathbf{x}_i\}_{i=1}^N$, we define:

$$\mathbf{f} = (f(\mathbf{x}_1), \dots, f(\mathbf{x}_N)),$$

where $\mathbf{f}$ is an **N-dimensional vector of function values** evaluated by the **N input points** $\{\mathbf{x}_i\}_{i=1}^N$.

Optional: Gaussian Processes

A GP possesses the properties of Gaussian distribution, is can be parameterised by a **mean function**, $\mu(\mathbf{x})$, and a **covariance function (or kernel)**, $K(\mathbf{x}, \mathbf{x}')$.

For a 2-dimensional case:

$$p(f(\mathbf{x}_1), f(\mathbf{x}_2)) = \mathcal{N}(\mu_g, \Sigma_g),$$

where

$$\mu_g = \mathcal{N}\left(\begin{bmatrix} \mu(\mathbf{x}_1) \\ \mu(\mathbf{x}_2) \end{bmatrix}\right), \quad \Sigma_g = \mathcal{N}\left(\begin{bmatrix} K(\mathbf{x}_1, \mathbf{x}_1) & K(\mathbf{x}_1, \mathbf{x}_2) \\ K(\mathbf{x}_2, \mathbf{x}_1) & K(\mathbf{x}_2, \mathbf{x}_2) \end{bmatrix}\right),$$

For **N-dimensional case**, μ_g is an $N \times 1$ column vector and Σ_g is a $N \times N$ matrix.

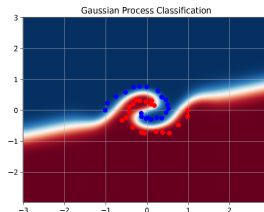
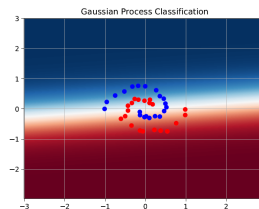
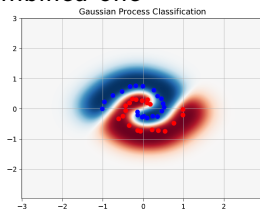
Optional: kernels

$$K(\mathbf{x}_i, \mathbf{x}_j) = \alpha \exp \left(- \left(\frac{\|\mathbf{x}_i - \mathbf{x}_j\|}{r^2} \right)^\beta \right)$$

- The kernel contains parameters $\theta = (\alpha, \beta, r)$.
- The unknown kernel parameters θ can be learned from the data!
- Once the mean and covariance functions are determined, a GP simply follows the rules applied to a multivariate Gaussian.

Optional: kernels

- Kernel design is essential for GP performance
- Kernels can be combined
- Below are the Squared Exponential kernel, Linear kernel, and the combined one



Reading list

BIS Christopher Bishop. *Pattern Recognition and Machine Learning*, Chapter 11